

In computer science, **counting sort** is an algorithm for sorting a collection of objects according to keys that are small positive integers; that is, it is an integer sorting algorithm. It operates by counting the number of objects that possess distinct key values, and applying prefix sum on those counts to determine the positions of each key value in the output sequence. Its running time is linear in the number of items and the difference between the maximum key value and the minimum key value, so it is only suitable for direct use in situations where the variation in keys is not significantly greater than the number of items. It is often used as a subroutine in radix sort, another sorting algorithm, which can handle larger keys more efficiently.^{[1][2][3]}

Counting sort is not a comparison sort; it uses key values as indexes into an array and the $\Omega(n \log n)$ lower bound for comparison sorting will not apply.^[1] Bucket sort may be used in lieu of counting sort, and entails a similar time analysis. However, compared to counting sort, bucket sort requires linked lists, dynamic arrays, or a large amount of pre-allocated memory to hold the sets of items within each bucket, whereas counting sort stores a single number (the count of items) per bucket.^[4]

Input and output assumptions

In the most general case, the input to counting sort consists of a collection of n items, each of which has a non-negative integer key whose maximum value is at most k .^[3] In some descriptions of counting sort, the input to be sorted is assumed to be more simply a sequence of integers itself,^[1] but this simplification does not accommodate many applications of counting sort. For instance, when used as a subroutine in radix sort, the keys for each call to counting sort are individual digits of larger item keys; it would not suffice to return only a sorted list of the key digits, separated from the items.

In applications such as in radix sort, a bound on the maximum key value k will be known in advance, and can be assumed to be part of the input to the algorithm. However, if the value of k is not already known then it may be computed, as a first step, by an additional loop over the data to determine the maximum key value.

The output is an array of the elements ordered by their keys. Because of its application to radix sorting, counting sort must be a stable sort; that is, if two elements share the same key, their relative order in the output array and their relative order in the input array should match.^{[1][2]}

Pseudocode

In pseudocode, the algorithm may be expressed as:

```
function CountingSort(input, k) is
    count ← array of k + 1 zeros
    output ← array of same length as input

    for i = 0 to length(input) - 1 do
        j = key(input[i])
        count[j] = count[j] + 1

    for i = 1 to k do
        count[i] = count[i] + count[i - 1]

    for i = length(input) - 1 down to 0 do
        j = key(input[i])
        count[j] = count[j] - 1
```

```
        output[count[j]] = input[i]

    return output
```

Where **input** is the array to be sorted, **key** returns the numeric key of each item in the input array, **count** is an auxiliary array used first to store the numbers of items with each key, and then (after the second loop) to store the positions where items with each key should be placed, **k** is the maximum value of the non-negative key values and **output** is the sorted output array.

In summary, the algorithm loops over the items in the first loop, computing a histogram of the number of times each key occurs within the **input** collection. After that in the second loop, it performs a prefix sum computation on **count** in order to determine, for each key, the position range where the items having that key should be placed; i.e. items of key **i** should be placed starting in position **count[i]**. Finally, in the third loop, it loops over the items of **input** again, but in reverse order, moving each item into its sorted position in the **output** array.^{[1][2][3]}

The relative order of items with equal keys is preserved here; i.e., this is a stable sort.

Complexity analysis

Because the algorithm uses only simple **for** loops, without recursion or subroutine calls, it is straightforward to analyze. The initialization of the count array, and the second for loop which performs a prefix sum on the count array, each iterate at most $k + 1$ times and therefore take $O(k)$ time. The other two for loops, and the initialization of the output array, each take $O(n)$ time. Therefore, the time for the whole algorithm is the sum of the times for these steps, $O(n + k)$.^{[1][2]}

Because it uses arrays of length $k + 1$ and n , the total space usage of the algorithm is also $O(n + k)$.^[1] For problem instances in which the maximum key value is significantly smaller than the number of items, counting sort can be highly space-efficient, as the only storage it uses other than its input and output arrays is the Count array which uses space $O(k)$.^[5]

Variant algorithms

If each item to be sorted is itself an integer, and used as key as well, then the second and third loops of counting sort can be combined; in the second loop, instead of computing the position where items with key **i** should be placed in the output, simply append **Count[i]** copies of the number **i** to the output.

This algorithm may also be used to eliminate duplicate keys, by replacing the **Count** array with a bit vector that stores a **one** for a key that is present in the input and a **zero** for a key that is not present. If additionally the items are the integer keys themselves, both second and third loops can be omitted entirely and the bit vector will itself serve as output, representing the values as offsets of the non-**zero** entries, added to the range's lowest value. Thus the keys are sorted and the duplicates are eliminated in this variant just by being placed into the bit array.

For data in which the maximum key size is significantly smaller than the number of data items, counting sort may be parallelized by splitting the input into subarrays of approximately equal size, processing each subarray in parallel to generate a separate count array for each subarray, and then merging the count arrays. When used as part of a parallel radix sort algorithm, the key size (base of the radix representation)

should be chosen to match the size of the split subarrays.^[6] The simplicity of the counting sort algorithm and its use of the easily parallelizable prefix sum primitive also make it usable in more fine-grained parallel algorithms.^[7]

As described, counting sort is not an in-place algorithm; even disregarding the count array, it needs separate input and output arrays. It is possible to modify the algorithm so that it places the items into sorted order within the same array that was given to it as the input, using only the count array as auxiliary storage; however, the modified in-place version of counting sort is not stable.^[3]